Instituto Politécnico Nacional
Escuela Superior de Cómputo

WEB CLIENT AND BACKEND DEVELOPMENT FRAMEWORKS

Práctica: Descarga, creación y ejecución de imagen Docker del minisitio de nuevo ingreso de ESCOM

Profesor: *Dr. José Asunción Enríquez Zárate*

Alumno: *Vanesa Rodriguez Verdin Sandoval*

Boleta: *202263188*

Grupo: *7CM2*

18 de septiembre de 2026

Índice

1. Introducción **1**

2. Conceptos teóricos **1**

3. Desarrollo **1**

3.1. Descarga del contenido del minisitio 1
3.2. Los Dockerfiles 2
3.3. Construcción de las imágenes (`docker build`) 3
3.4. Ejecución de los contenedores (`docker run`) 4

4. Resultados **4**

5. Conclusiones **5**

Índice de figuras

1. Contenido descargado del minisitio, dentro de `sitio/` 2
2. `docker build` de la imagen basada en Apache 3
3. `docker build` de la imagen basada en Nginx 3
4. Ambos contenedores corriendo simultáneamente (`docker ps`) 4
5. Minisitio funcionando en `http://localhost:8080`, servido por Apache 5
6. Minisitio funcionando en `http://localhost:8081`, servido por Nginx 5

1. Introducción

Un sitio estático no necesita más que un servidor web y sus propios archivos para funcionar, pero «funcionar en mi máquina» y «funcionar donde sea» son dos problemas distintos: el segundo exige que el entorno completo (el sistema operativo, el servidor y su configuración) viaje junto con el contenido. Docker resuelve eso empaquetando ambas cosas en una sola unidad ejecutable, la imagen, de la que se pueden levantar tantas instancias en ejecución (los contenedores) como se necesiten, todas idénticas entre sí y a como se probaron en desarrollo.

Esta práctica toma el minisitio de *Nuevo Ingreso ESCOM* [1], ya descargado íntegro para la Tarea 2 de esta materia, y lo empaqueta dos veces: una imagen basada en Apache httpd y otra basada en Nginx, cada una servida en un puerto distinto de la misma máquina. El objetivo no es solo lograr que el sitio cargue, sino evidenciar el mecanismo completo: construir la imagen a partir de un `Dockerfile`, ejecutar el contenedor mapeando su puerto interno hacia uno del anfitrión, y verificar en el navegador que el mismo contenido responde igual sin importar cuál de los dos servidores lo sirva.

2. Conceptos teóricos

Imagen Docker. Una imagen es una plantilla de solo lectura que contiene un sistema de archivos mínimo, un runtime (o un servidor, en este caso) y todo lo que la aplicación necesita para correr, construida en capas apiladas donde cada instrucción del `Dockerfile` agrega una capa nueva encima de la anterior [2]. La imagen no se ejecuta por sí sola: es el molde a partir del cual se crean los contenedores.

Contenedor. Un contenedor es una instancia en ejecución de una imagen: un proceso aislado (con su propio sistema de archivos, red y espacio de procesos, aunque comparte el kernel del sistema operativo anfitrión) que arranca en segundos porque no tiene que iniciar un sistema operativo completo, a diferencia de una máquina virtual [2]. De una misma imagen pueden arrancarse varios contenedores simultáneos, cada uno aislado de los demás.

Dockerfile. Es el archivo de texto plano que describe, instrucción por instrucción, cómo construir una imagen: de qué imagen base parte (`FROM`), qué archivos se copian dentro de ella (`COPY`), qué puerto declara que usará (`EXPOSE`), entre otras [3]. Es la receta versionable de la imagen: el mismo `Dockerfile` produce siempre la misma imagen.

Mapeo de puertos. Un contenedor vive en su propia red aislada del anfitrión, así que un servidor que escucha en el puerto 80 *dentro* del contenedor es invisible desde afuera hasta que se publica explícitamente un puerto del anfitrión hacia él. Eso es lo que hace la bandera `-p host:contenedor` de `docker run`: crea un reenvío desde un puerto de la máquina real hacia el puerto interno del contenedor, de modo que `-p 8080:80` pone lo que el contenedor sirve en el 80 disponible en `http://localhost:8080` del anfitrión.

3. Desarrollo

3.1. Descarga del contenido del minisitio

El sitio se descargó completo (HTML, hojas de estilo, JavaScript e imágenes) con una herramienta de reflejo de sitios que sigue los enlaces internos y reescribe las rutas para que el resultado funcione sin conexión:

Código 1: Descarga completa del minisitio con wget

```
wget --mirror --convert-links --adjust-extension \
     --page-requisites --no-parent \
     https://www.escom.ipn.mx/nuevoingreso27_1/
```

El resultado se organizó dentro de la carpeta `sitio/` de esta práctica, reutilizando el mismo contenido ya descargado para la Tarea 2 de la materia: es el mismo sitio origen, así que no tenía sentido descargarlo dos veces. La carpeta contiene `index.html`, la hoja `skin.css`, el paquete de estilos `themekit/` y los recursos (PDF e imágenes) en `media/`.

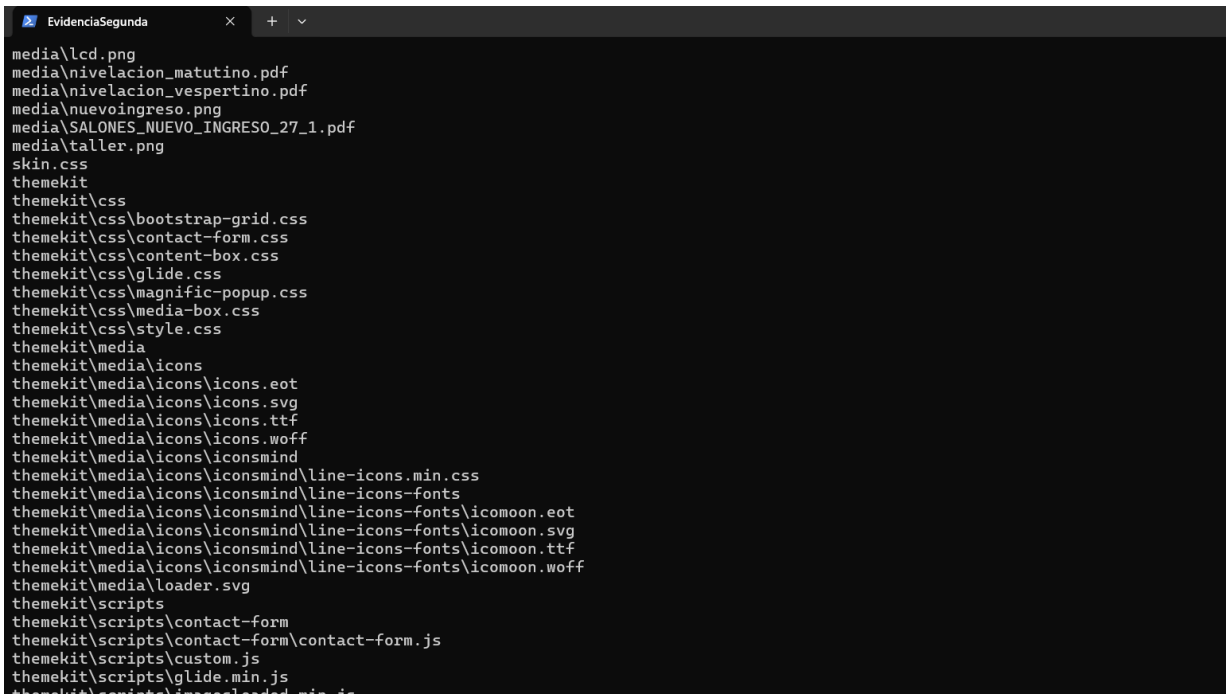


Figura 1: Contenido descargado del minisitio, dentro de `sitio/`

3.2. Los Dockerfiles

Se escribieron dos `Dockerfile` distintos que comparten el mismo contenido (`sitio/`) pero cada uno lo sirve con un servidor web diferente. La única diferencia real entre ambos es la carpeta de destino del `COPY`, que depende de dónde cada imagen oficial espera su *document root*.

Código 2: Dockerfile.apache: imagen basada en httpd

```
1 FROM httpd:2.4-alpine
2
3 COPY sitio/ /usr/local/apache2/htdocs/
4
5 EXPOSE 80
```

- `FROM httpd:2.4-alpine` parte de la imagen oficial de Apache HTTP Server 2.4 sobre Alpine Linux, una distribución mínima que mantiene la imagen ligera [4].
- `COPY sitio/ /usr/local/apache2/htdocs/` copia el sitio dentro del *document root* por defecto de esa imagen.
- `EXPOSE 80` documenta que el proceso dentro del contenedor escucha en el puerto 80; es informativo y no publica el puerto por sí solo, eso lo hace `-p` en `docker run`.

Código 3: Dockerfile.nginx: imagen basada en nginx

```
1 FROM nginx:alpine
2
3 COPY sitio/ /usr/share/nginx/html/
4
5 EXPOSE 80
```

- `FROM nginx:alpine` parte de la imagen oficial de Nginx sobre Alpine [5].
- `COPY sitio/ /usr/share/nginx/html/` copia el mismo sitio, ahora en el *document root* por defecto de Nginx, distinto al de Apache y la única razón por la que se necesitan dos `Dockerfile` en vez de uno solo.
- `EXPOSE 80` tiene el mismo papel documental que en la imagen de Apache.

3.3. Construcción de las imágenes (`docker build`)

Ambas imágenes se construyen desde la misma carpeta de contexto (esta práctica), seleccionando el `Dockerfile` correspondiente con la bandera `-f` y etiquetándolas (`-t`) con un nombre memorable:

Código 4: Construcción de las dos imágenes

```
docker build -f Dockerfile.apache -t nuevoingreso-apache .
docker build -f Dockerfile.nginx  -t nuevoingreso-nginx  .
```

En ambos casos Docker procesa el `Dockerfile` instrucción por instrucción, descarga la imagen base si no está en caché local, y agrega una capa por cada instrucción hasta terminar con la imagen etiquetada y lista para ejecutarse.

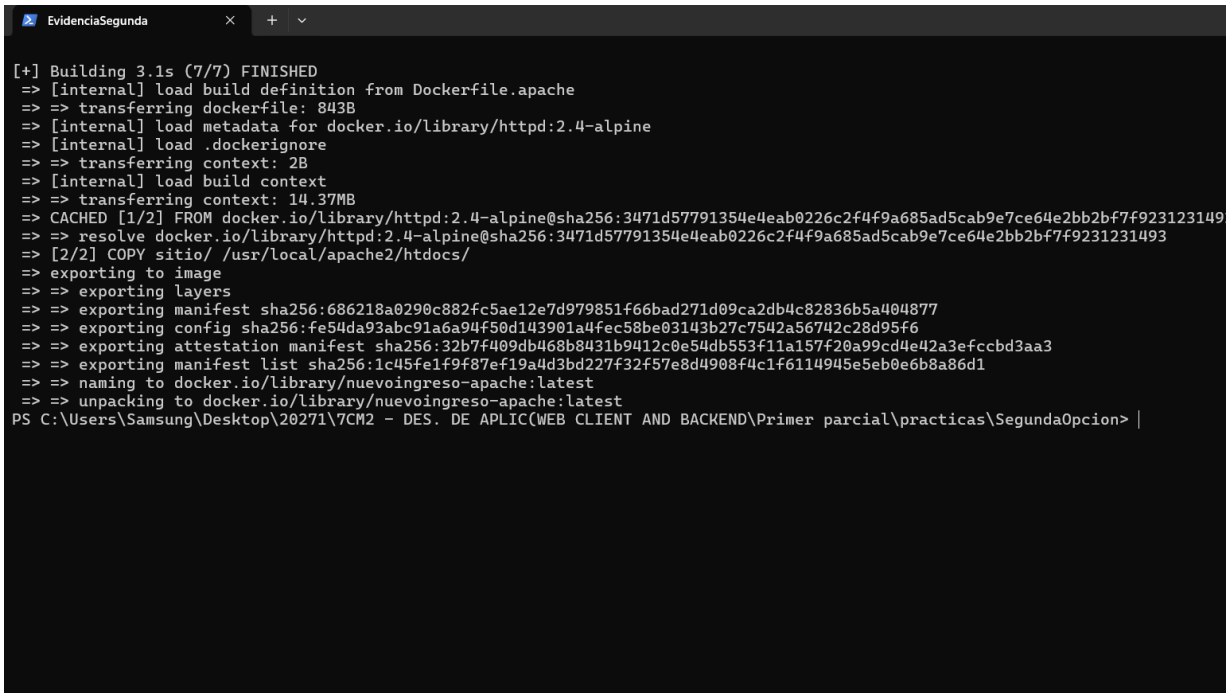


Figura 2: `docker build` de la imagen basada en Apache

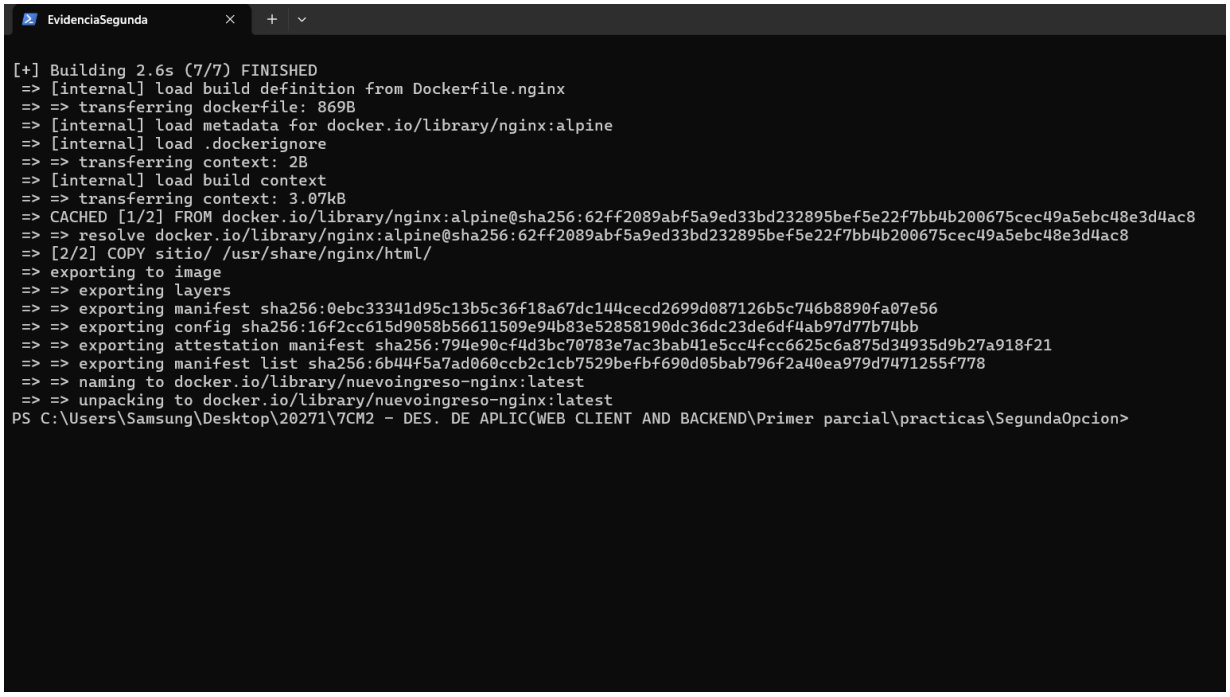


Figura 3: `docker build` de la imagen basada en Nginx

3.4. Ejecución de los contenedores (`docker run`)

Con las dos imágenes ya construidas, se levanta un contenedor de cada una, publicando puertos distintos del anfitrión hacia el puerto 80 interno de cada contenedor para que ambos puedan convivir sin chocar:

Código 5: Ejecución de ambos contenedores

```
docker run --name nuevoingreso-apache -d -p 8080:80 nuevoingreso-apache
docker run --name nuevoingreso-nginx  -d -p 8081:80 nuevoingreso-nginx

docker ps
```

La bandera `-d` corre el contenedor en segundo plano, `-name` le da un nombre legible en vez del identificador aleatorio por defecto, y `-p 8080:80`/`-p 8081:80` son los mapeos de puerto descritos en la sección de conceptos teóricos: el 8080 y el 8081 del anfitrión quedan enlazados al 80 de cada contenedor respectivamente.

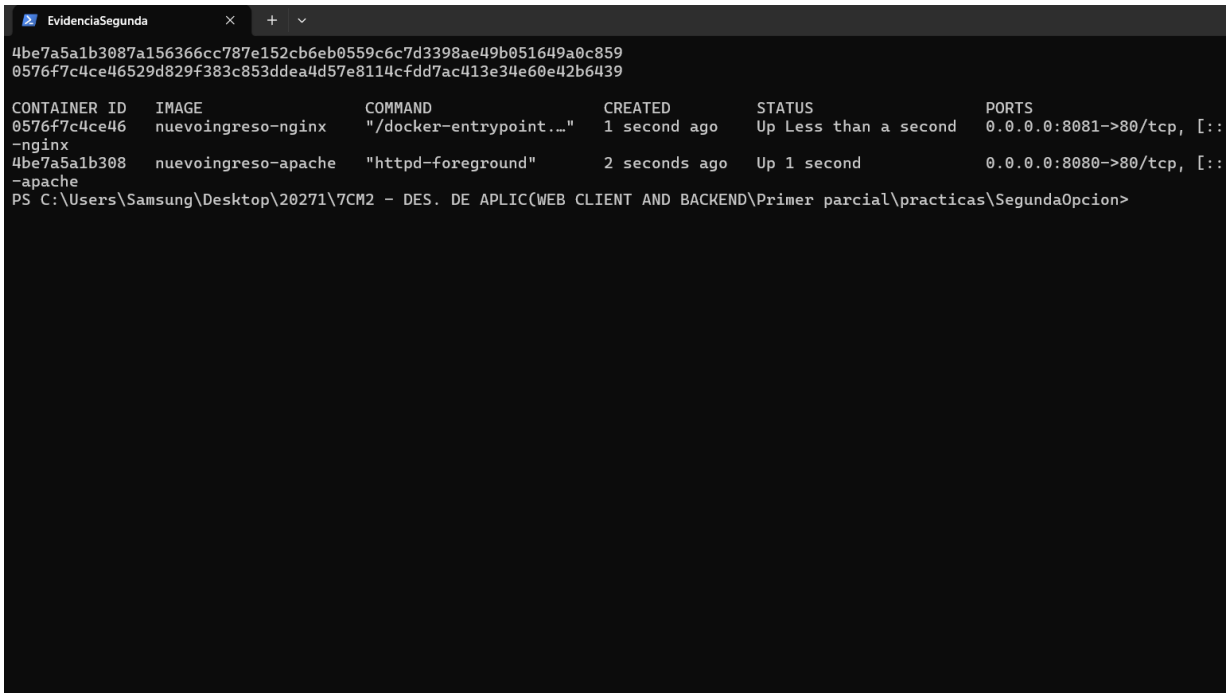


Figura 4: Ambos contenedores corriendo simultáneamente (`docker ps`)

4. Resultados

Con los dos contenedores arriba, el mismo minisitio responde en dos puertos distintos del anfitrión servido por dos motores distintos:

Servidor	Imagen base	Document root	URL
Apache httpd	`httpd:2.4-alpine`	`/usr/local/apache2/htdocs/`	`http://localhost:8080`
Nginx	`nginx:alpine`	`/usr/share/nginx/html/`	`http://localhost:8081`

Tabla 1: Las dos instancias del mismo sitio, una por servidor

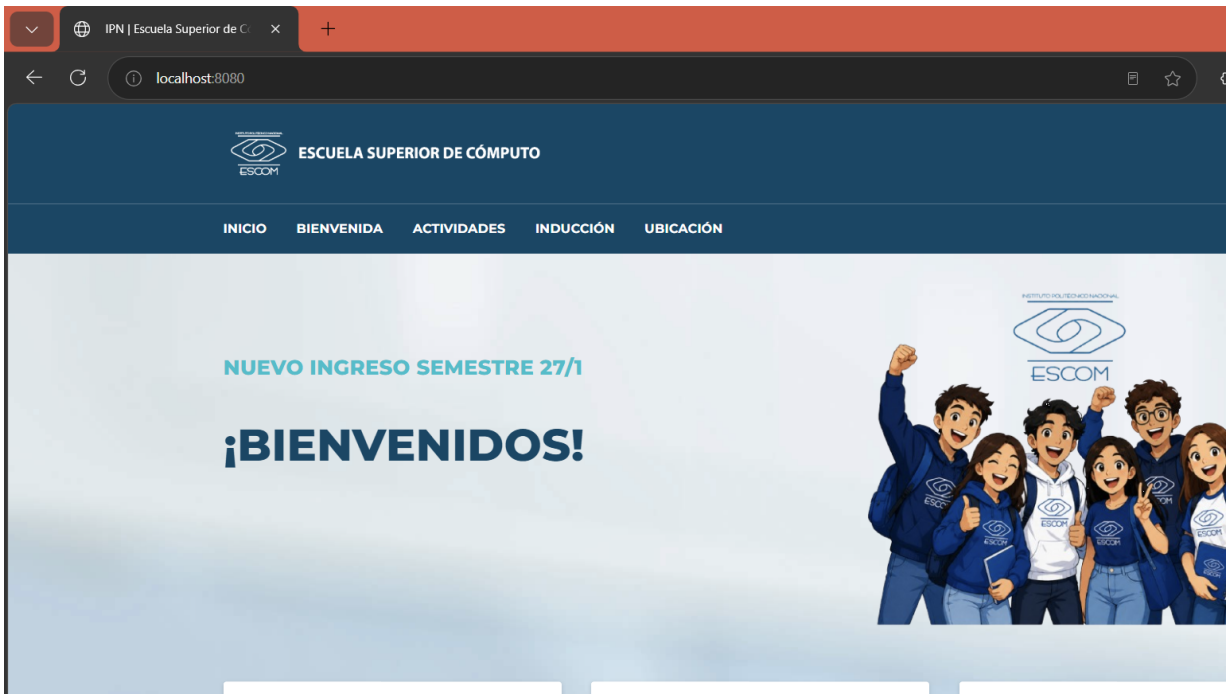


Figura 5: Minisitio funcionando en `http://localhost:8080`, servido por Apache

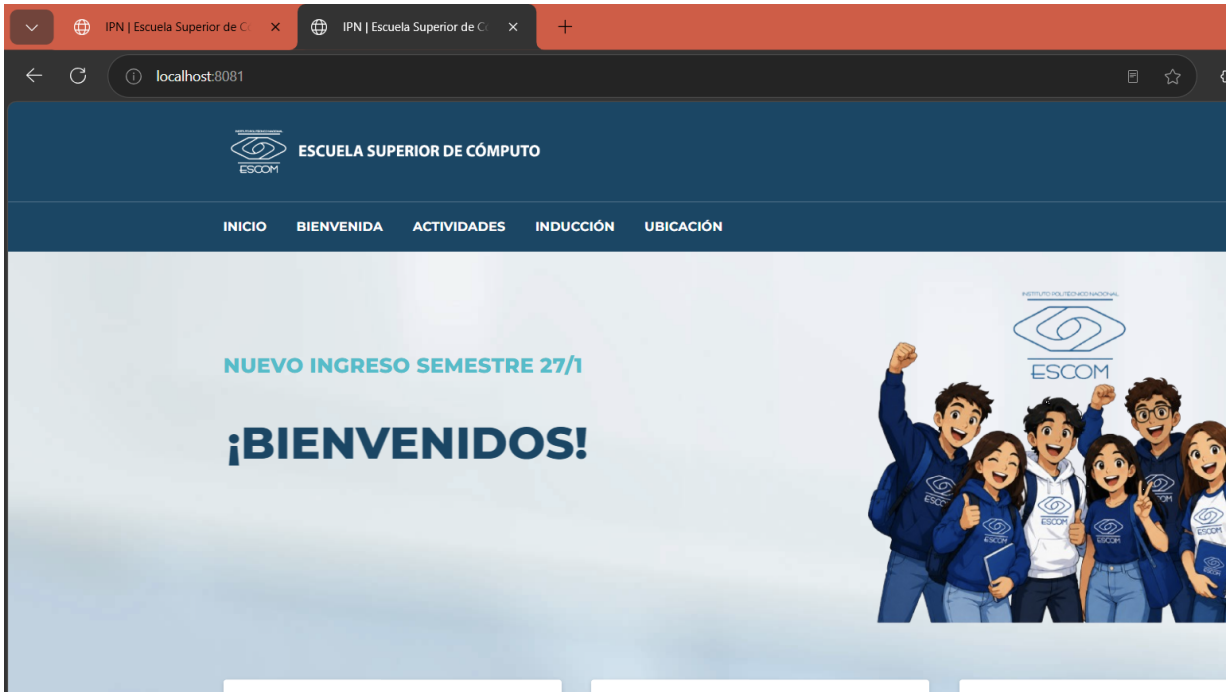


Figura 6: Minisitio funcionando en `http://localhost:8081`, servido por Nginx

Las dos capturas muestran el mismo contenido con la misma apariencia, lo cual confirma que el sitio no depende de ninguna característica propia de un servidor en particular: es HTML y CSS estático, así que a cualquiera de los dos motores les basta con entregar los archivos tal cual.

5. Conclusiones

Apache y Nginx no coinciden en dónde esperan encontrar los archivos a servir (`/usr/local/apache2/htdocs/` contra `/usr/share/nginx/html/`), y esa diferencia fue la dificultad real de la práctica: copiar el mismo `sitio/` a la ruta equivocada produce la página por defecto del servidor en vez del minisitio. Confirmar el *document root*

de cada imagen oficial antes de escribir el `COPY` evitó ese tropiezo. La segunda dificultad, más práctica que conceptual, fue mantener los puertos publicados sin choques: al correr ambos contenedores sobre el mismo anfitrión, publicar los dos en el mismo puerto externo (por ejemplo, 8080 para los dos) hace que el segundo `docker run` falle porque el puerto ya está tomado, de ahí que cada contenedor use un puerto distinto del anfitrión aunque los dos escuchen internamente en el 80.

Más allá de esos dos detalles, el ejercicio deja clara la separación de responsabilidades que ofrece Docker: la misma carpeta de contenido, sin modificar una sola línea, se sirve igual de bien desde dos servidores completamente distintos con solo cambiar el `Dockerfile` que la empaqueta. Esa portabilidad (contenido separado del servidor que lo entrega, y servidor separado del sistema operativo que lo hospeda) es exactamente el problema que las arquitecturas contenedorizadas de la Unidad V de esta materia dan por resuelto, y que aquí se comprobó con un caso mínimo.

[1] Escuela Superior de Cómputo, IPN. *Nuevo Ingreso ESCOM 2027-1*. URL: https://www.escom.ipn.mx/nuevoingreso27_1/ (visitado 18-09-2026).

[2] Docker Inc. *Docker overview*. URL: https://docs.docker.com/get-started/docker-overview/ (visitado 18-09-2026).

[3] Docker Inc. *Dockerfile reference*. URL: https://docs.docker.com/reference/dockerfile/ (visitado 18-09-2026).

[4] Apache Software Foundation. *httpd - Official Image*. URL: https://hub.docker.com/_/httpd (visitado 18-09-2026).

[5] F5, Inc. *nginx - Official Image*. URL: https://hub.docker.com/_/nginx (visitado 18-09-2026).